

Digital Design with Verilog

FSM Completeness and Correctness

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

**Once we specify FSM for a
problem/system**

=== >

**Implementation is not
difficult**

FSM

Completeness and Correctness

**Example and Material referred from book : Frank
Vahid, “Digital Design” Wiley India, 2011**

Set Theoretic Description

Moore Machine is an ordered quintuple

$$M o o r e = (S, I, O, \delta, \lambda)$$

where

$$S = \{s_1, s_2, \dots, s_n\}$$

$$I = \{i_1, i_2, \dots, i_m\}$$

$$O = \{o_1, o_2, \dots, o_l\}$$

$$\delta = S \times I \rightarrow S$$

$$\lambda = S \rightarrow O$$

S= finite set of state ($\neq \phi$)

I = finite set of inputs ($\neq \phi$)

O=finite set of output ($\neq \phi$)

δ =next state function

λ = output function

FSM

We often draw FSM graphically, known as *state diagram*

Can also use table (state table), or textual languages

FSM Completeness

- Finite State Machine Transition Function

$$\delta : S \times I \rightarrow S$$

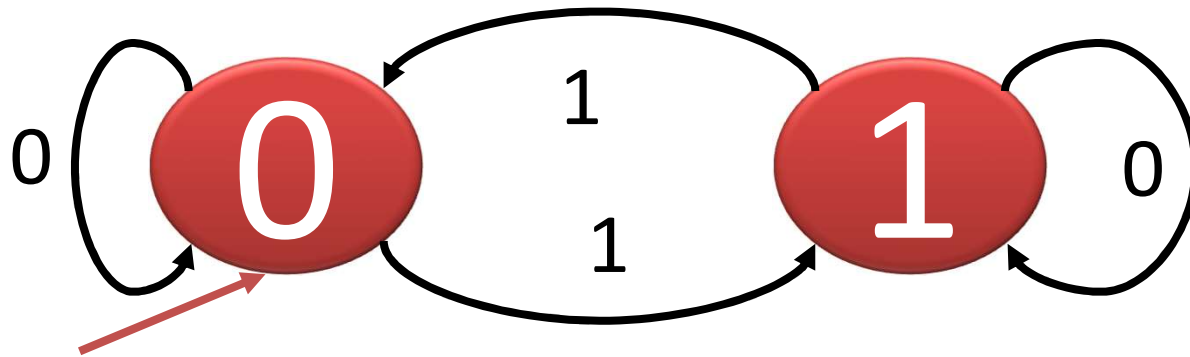
- δ : For all the states of FSM and For all the type of input
- FSM Completeness
 - If for all the states, all the input combination specified in transition function δ

FSM for T-FF: Completeness

State: 0, 1

Input : 0, 1

Output: 0,1



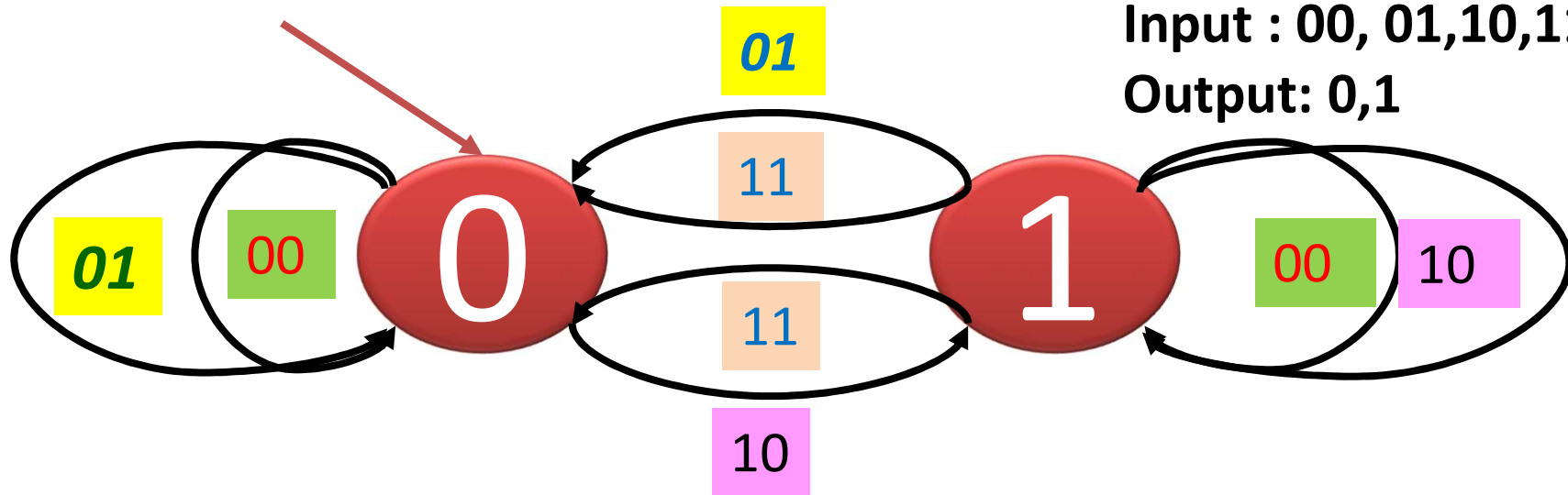
- Is this FSM complete? YES
- Number of Transition: $|S| \times |I| = 2 \times 2 = 4$
 - Plus one extra for initial transition shown in red

FSM for JK-FF: Completeness

State: 0, 1

Input : 00, 01, 10, 11

Output: 0, 1



- Is this FSM complete? YES
- Number of Transition: $|S| \times |I| = 2 \times 4 = 8$
 - Plus one extra for initial transition shown in red

FSM Correctness

- Finite State Machine Transition Function
$$\delta : S \times I \rightarrow S$$
- FSM is correct if For all the state of FSM
 - **AND operation of every pair of out going edges of a state =0**
 - *Meaning: for some condition FSM should not transition to more than one state*
 - **OR operation all out going edges of a state =1**
 - *Meaning: there should be at least one transition for every condition*

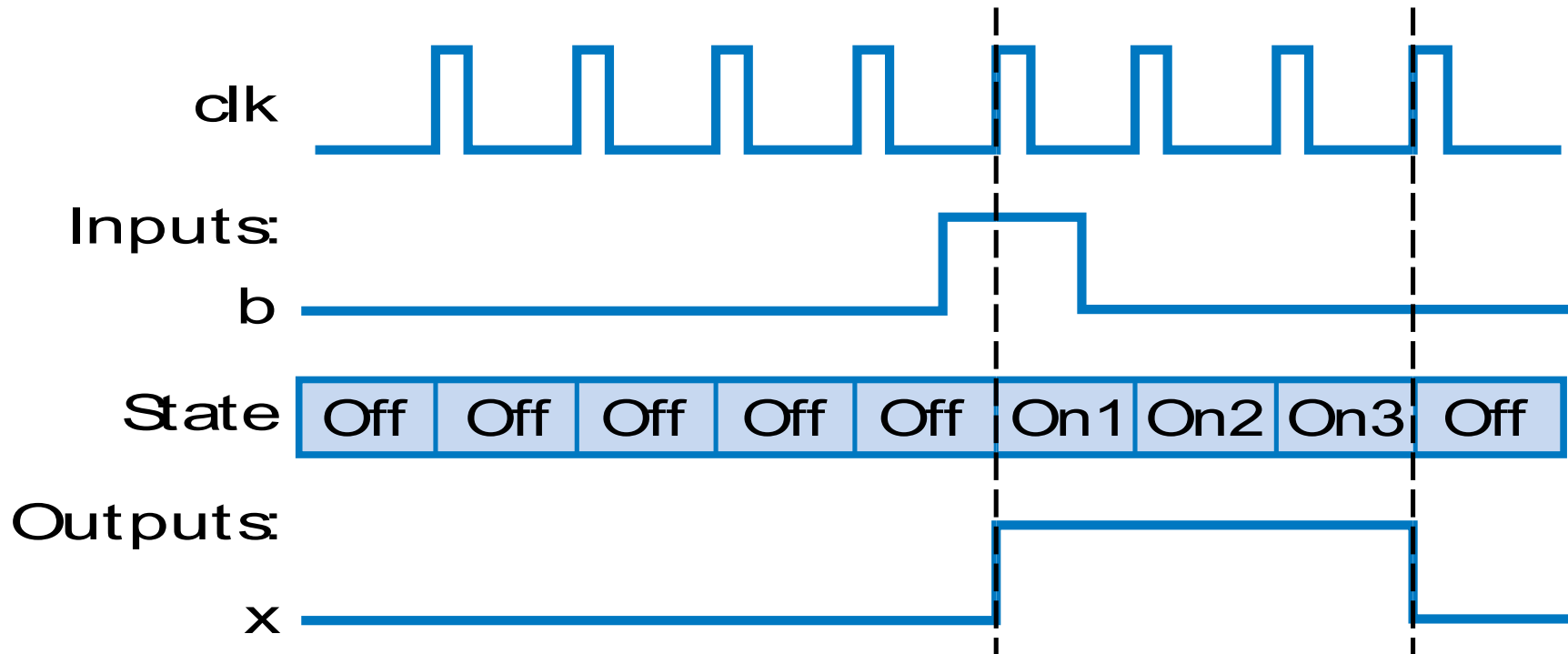
FSM Completeness Example A : Three-Cycles High Laser Timer

**Example and Material referred from book : Frank
Vahid, “Digital Design” Wiley India, 2011**

Extend FSM to Three-Cycles High Laser Timer

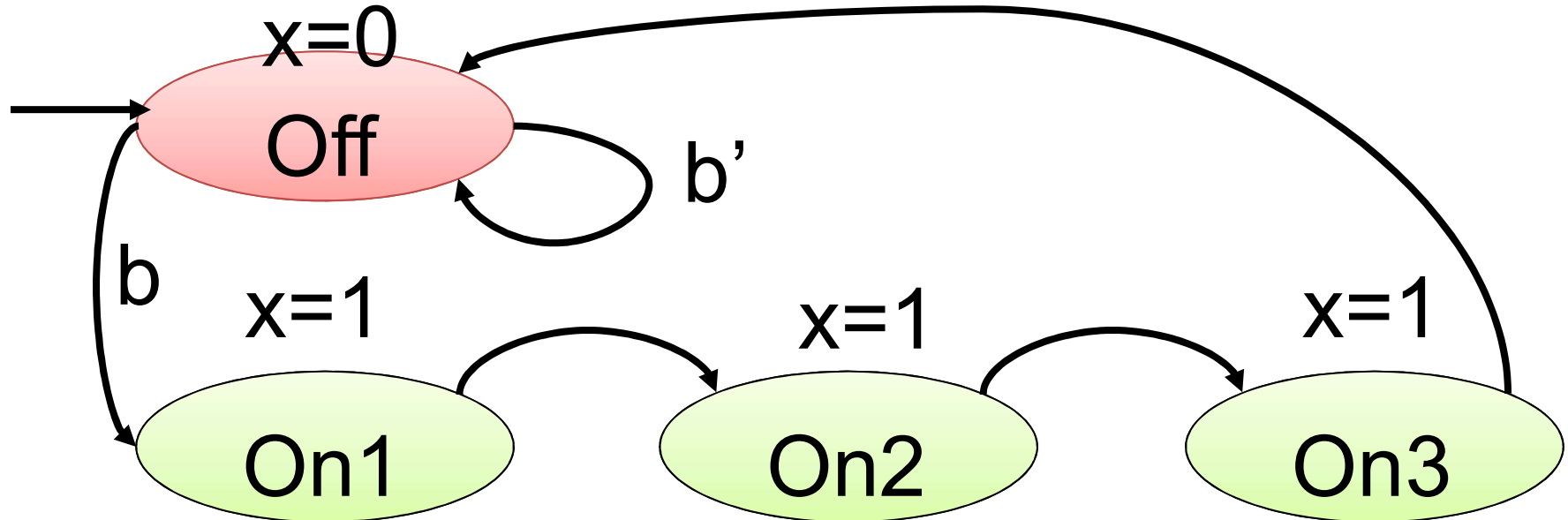
- Four states: Wait in “Off” state while b is 0 (b')
- When $b=1$ (& rising clock edge), transition to On1
 - Sets $X=1$
 - On next two clock edges, transition to On2, then On3, which also set $x=1$
- So $x=1$ for three cycles after button pressed

Extend FSM to Three-Cycles High Laser Timer



FSM of Three-Cycles High Laser Timer

Inputs: b ; Outputs: x

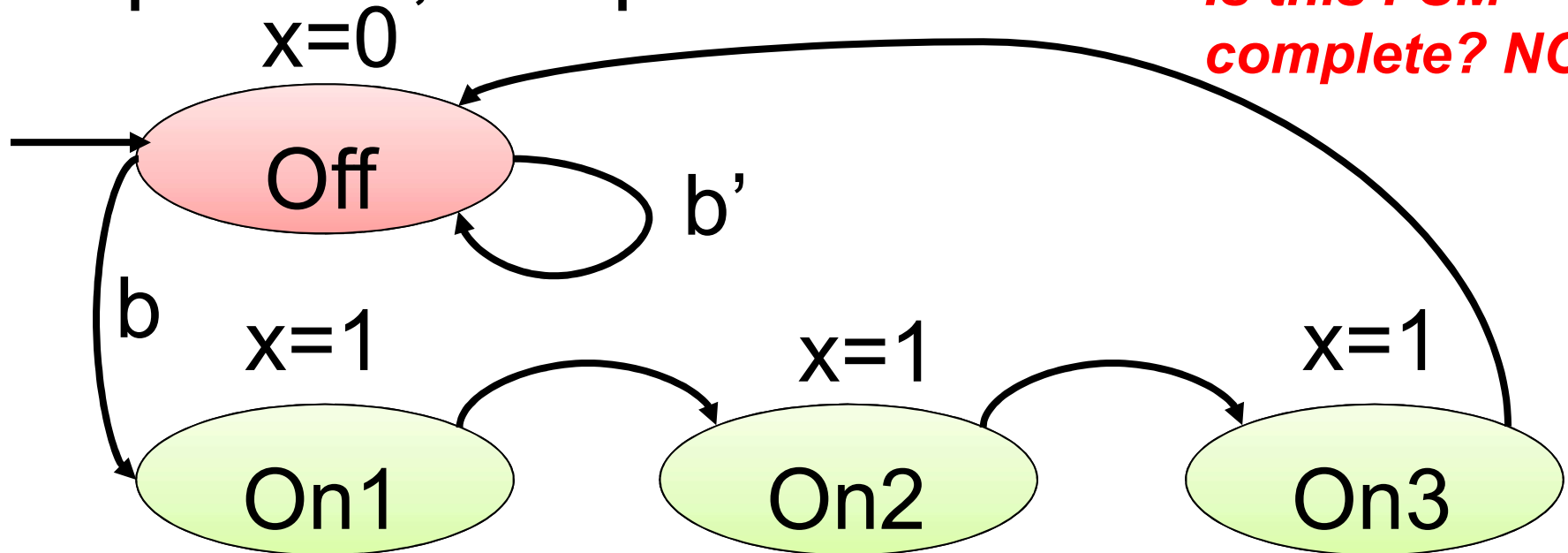


Note: Transition with no associated condition thus transistions to next state on next clock cycle

FSM of Three-Cycles High Laser Timer

Inputs: b ; Outputs: x

*Is this FSM
complete? NO*

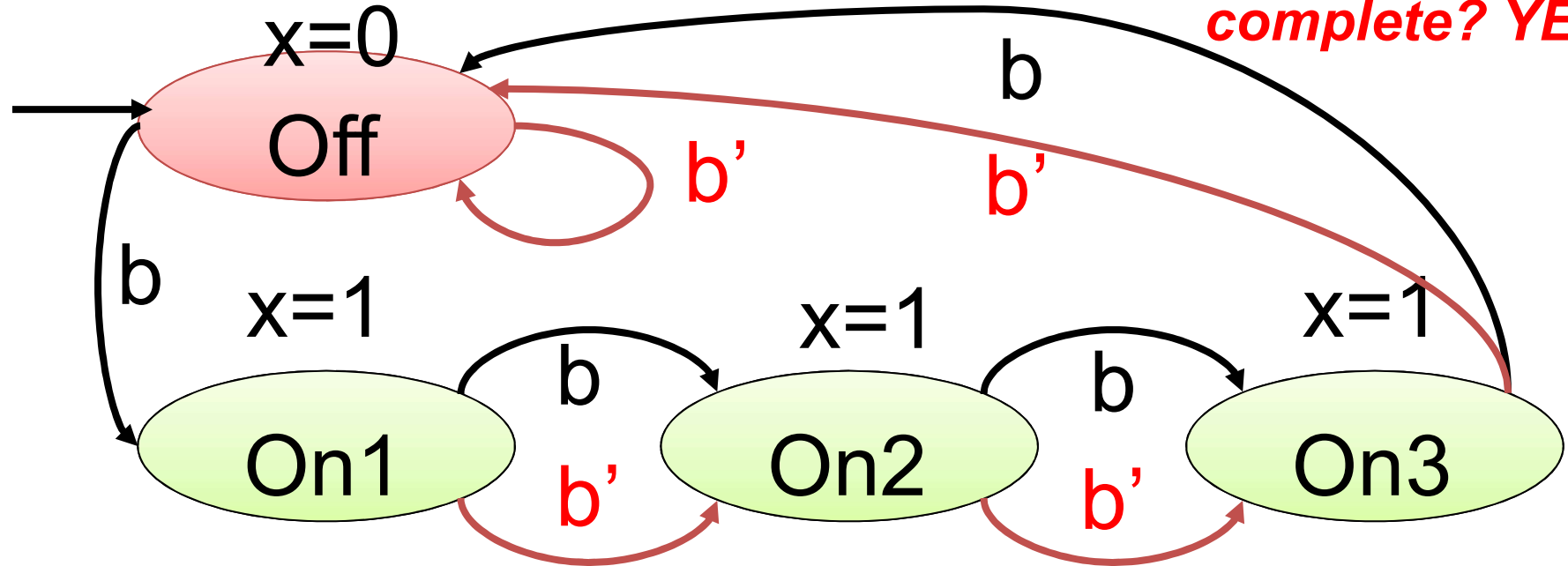


Note: Transition with no associated condition thus transistions to next state on next clock cycle

FSM of Three-Cycles High Laser Timer

Inputs: b ; Outputs: x

*Is this FSM
complete? YES*

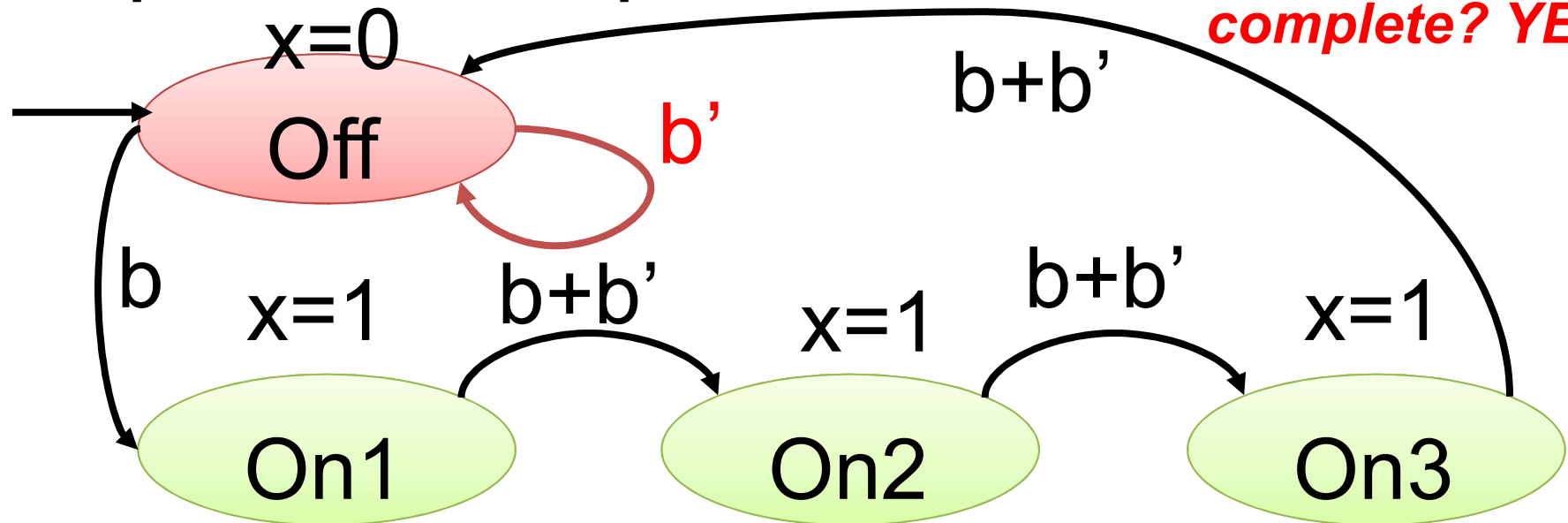


Note: Transition with no associated condition thus transistions to next state on next clock cycle

FSM of Three-Cycles High Laser Timer

Inputs: b ; Outputs: x

*Is this FSM
complete? YES*



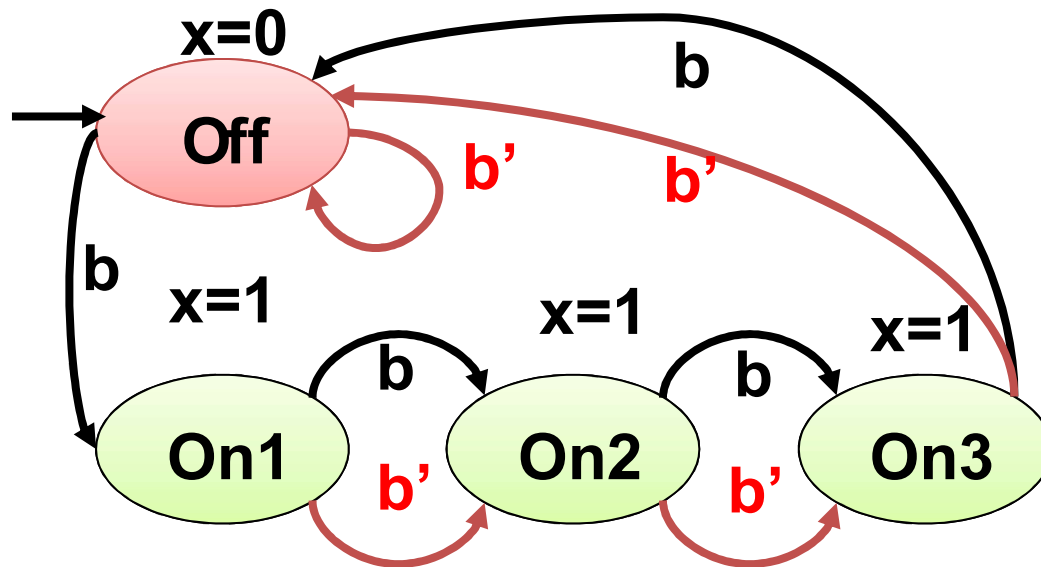
Note: Transition with no associated condition thus transistions to next state on next clock cycle

FSM Correctness Example A : Three-Cycles High Laser Timer

**Example and Material referred from book : Frank
Vahid, “Digital Design” Wiley India, 2011**

FSM Correctness

Inputs: b ; Outputs: x



AND operation every pair of out going edges of a state =0

$$b.b'=0$$

OR operation all out going edges of a state =1

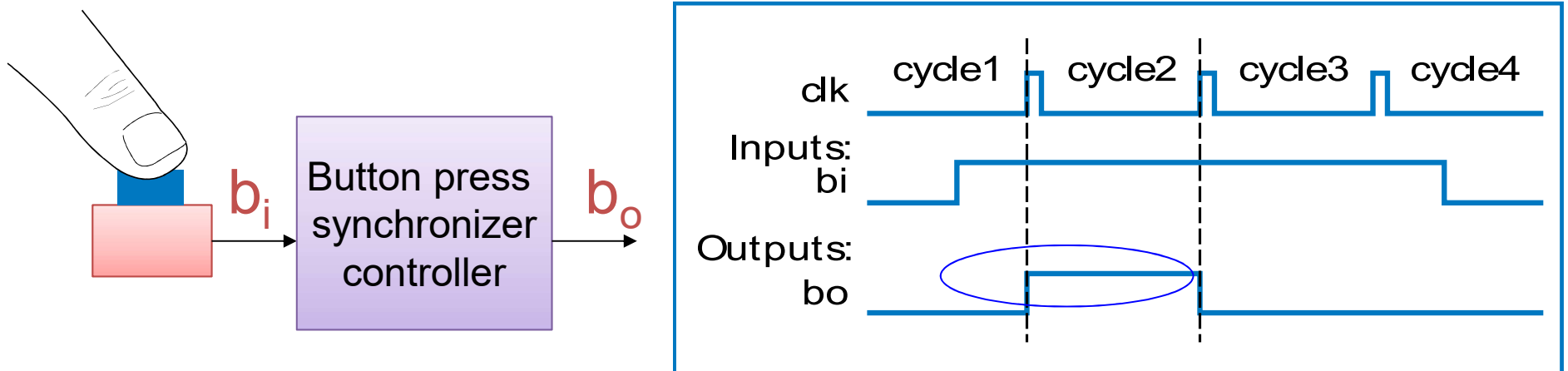
$$b+b'=1$$

Is this FSM is correctly specified based on rule?

FSM Completeness Example B: Button Press Synchronizer

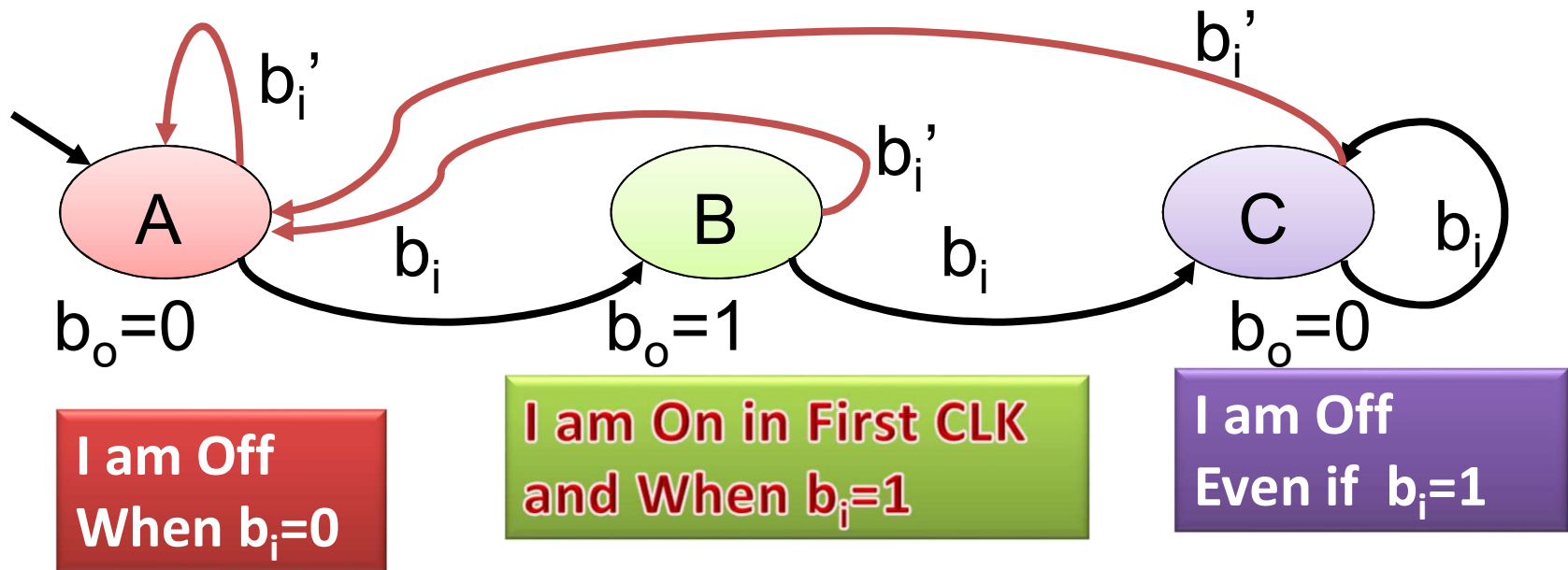
Button Press Synchronizer

- English Language Specification
- All most all the keyboards use this method
- We want simple sequential circuit
 - Converts button press to single cycle duration
 - Regardless of length of time that button actually pressed



FSM : Button Press Synchronizer

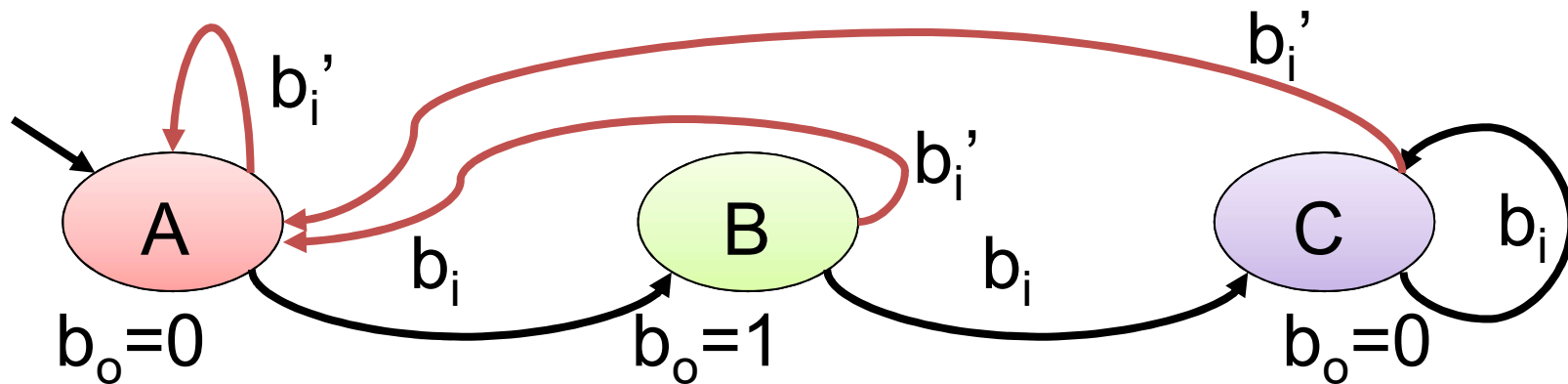
FSM inputs: b_i ; FSM outputs: b_o



Step 1: Design FSM

FSM : Button Press Synchronizer

FSM inputs: b_i ; FSM outputs: b_o



Is this FSM complete?

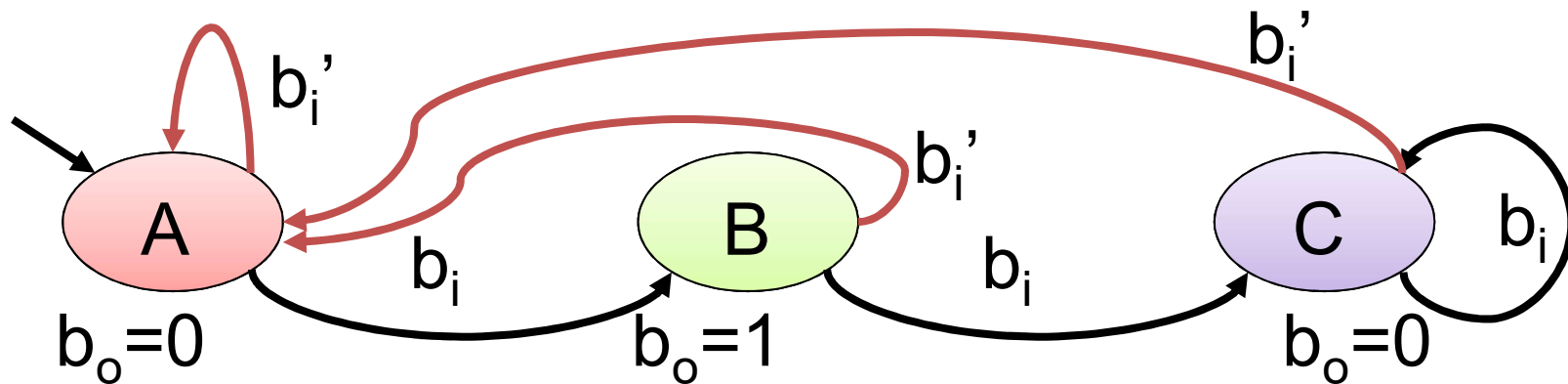
YES

Number of transition $\rightarrow |S| \times X + 1$

3 States, b_i can be 0,1 $\Rightarrow 3 \times 2 + 1 = 7$

FSM : Button Press Synchronizer

FSM inputs: b_i ; FSM outputs: b_o



Is this FSM correct?

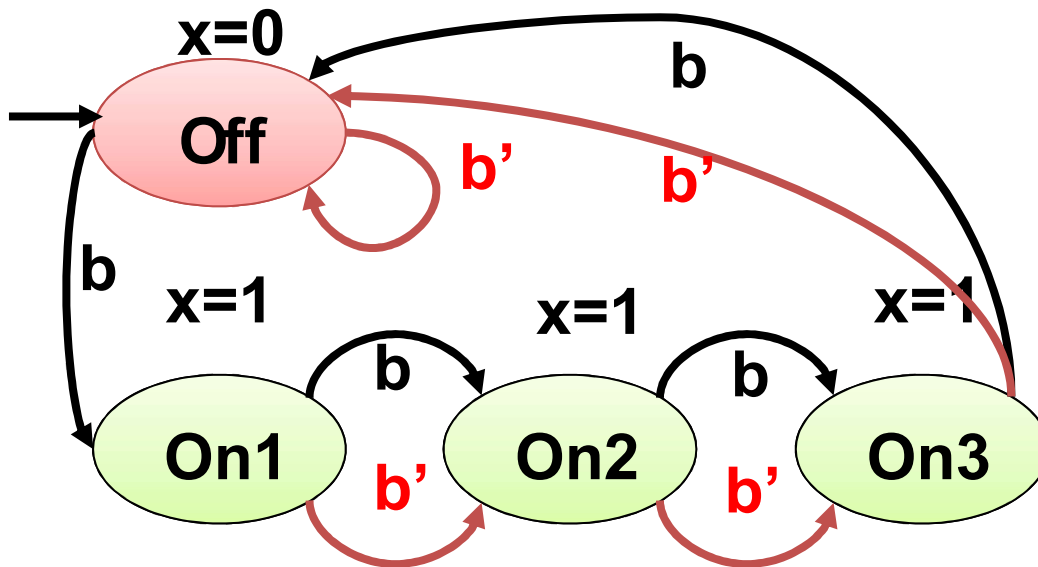
YES

For all the state exactly two out going transitions, one for b and other for b' : $b.b'=0$ $b+b'=1$

**FSM With Transition specified in Boolean
expression or Compressed form**

FSM Correctness

Inputs: b ; Outputs: x



AND operation
every pair of out
going edges of a
state =0

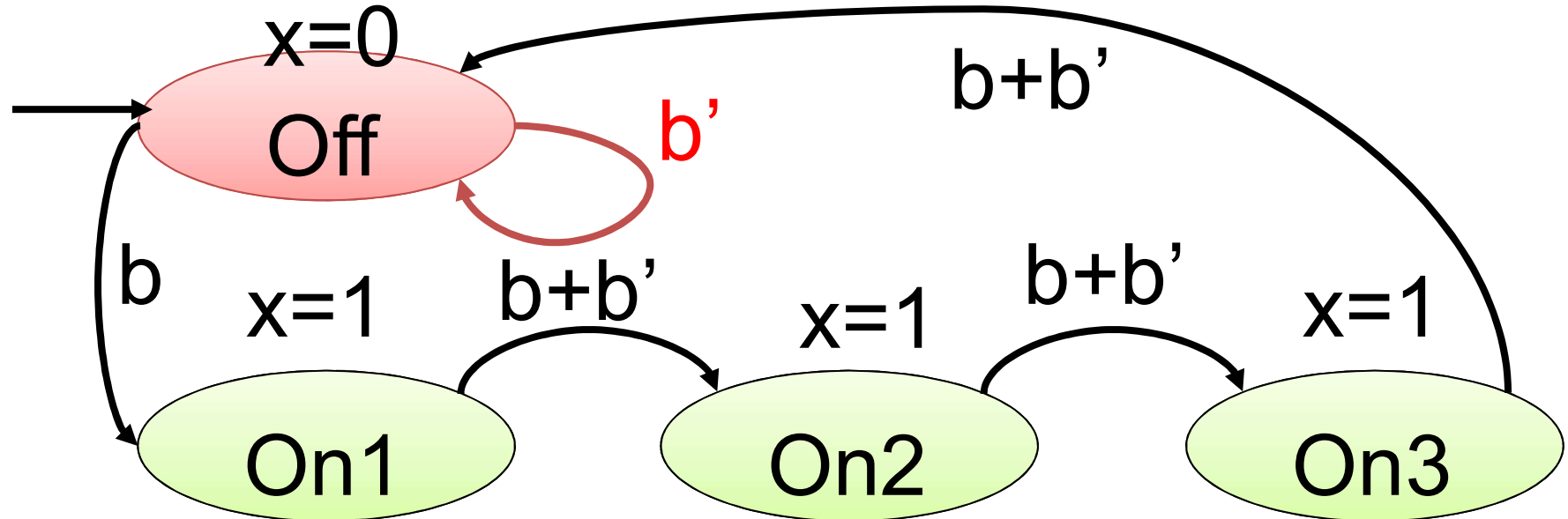
$$b.b'=0$$

OR operation all out
going edges of a
state =1

$$b+b'=1$$

FSM Completeness

Inputs: b ; Outputs: x

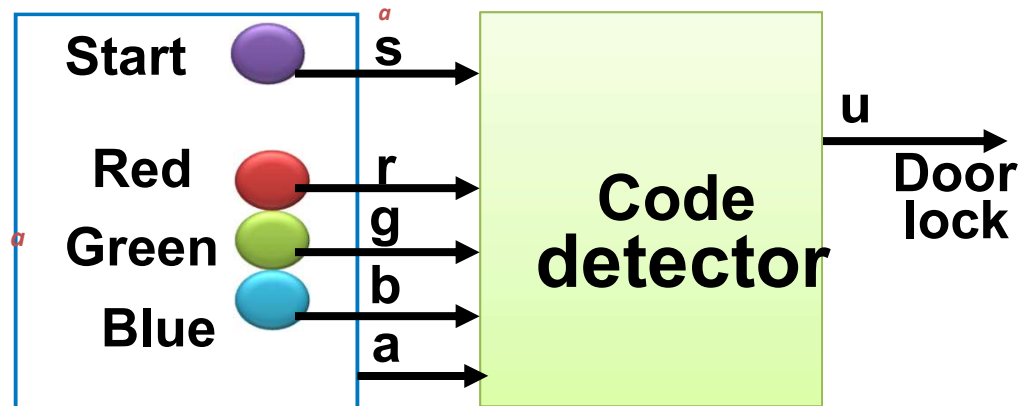


FSM Example : Code Detector

**Example and Material referred from book : Frank
Vahid, “Digital Design” Wiley India, 2011**

FSM Example : Code Detector

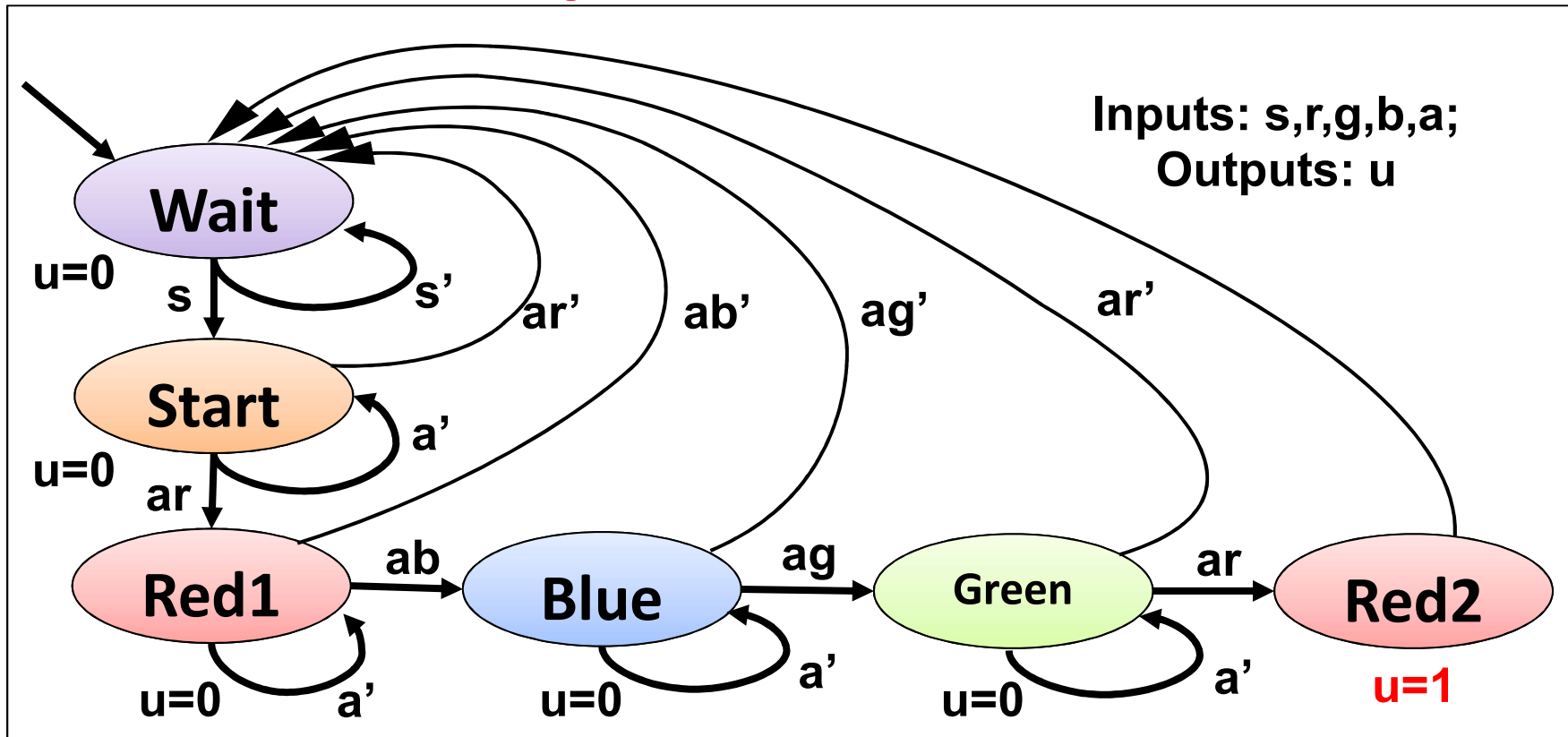
- Unlock door ($u=1$) only when buttons pressed in sequence:
 - **start, then red, blue, green, red**
- Input from each button: s, r, g, b
 - Also, output a indicates that some colored button pressed



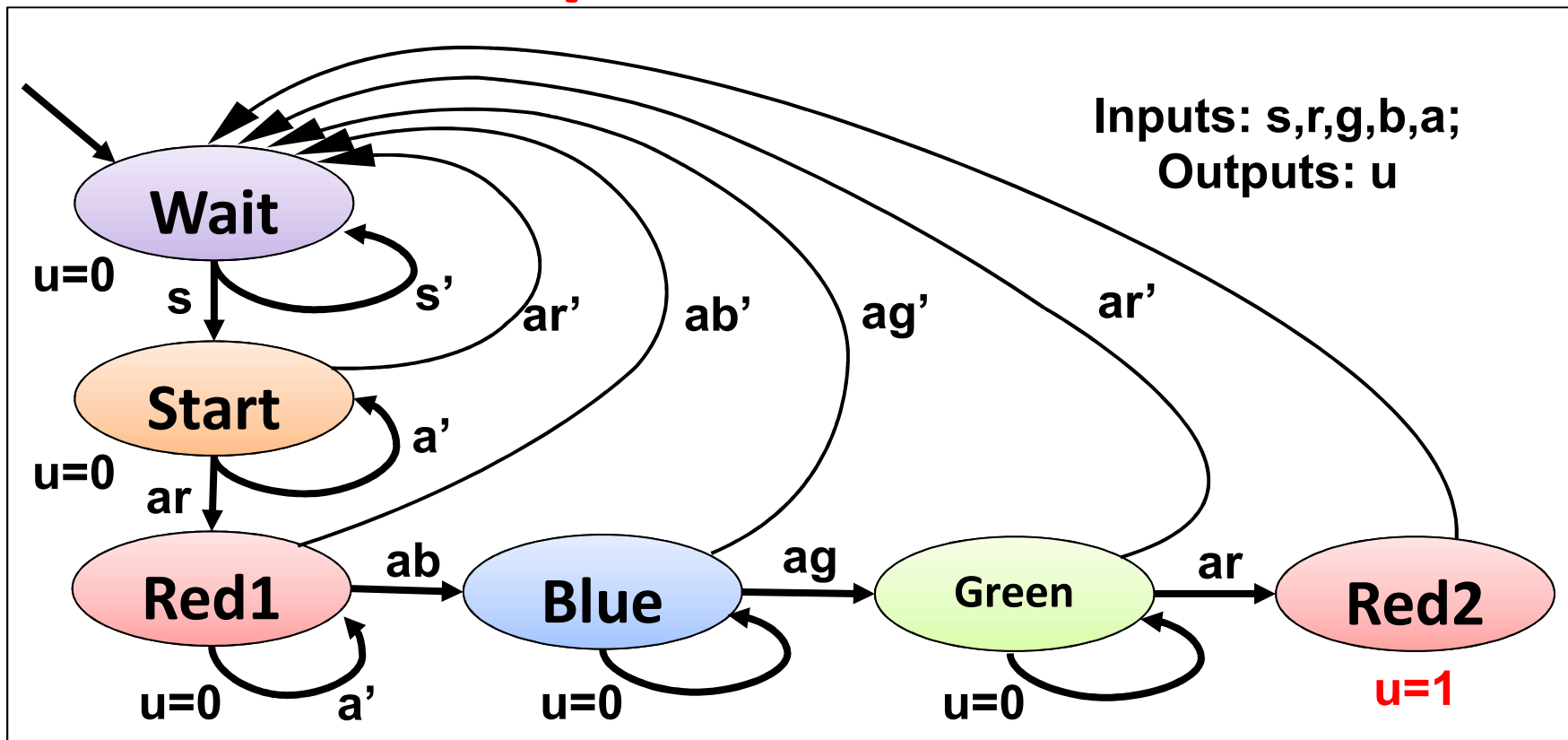
FSM Example : Code Detector

- Wait for start ($s=1$) in “Wait”,
- **Once started (“Start”)**
 - If see red, go to “Red1”
 - Then, if see blue, go to “Blue”, Then, if see green, go to “Green”, Then, if see red, go to “Red2”
 - In that state, open the door ($u=1$)
 - Wrong button at any step, return to “Wait”

FSM Example 11 : Code Detector

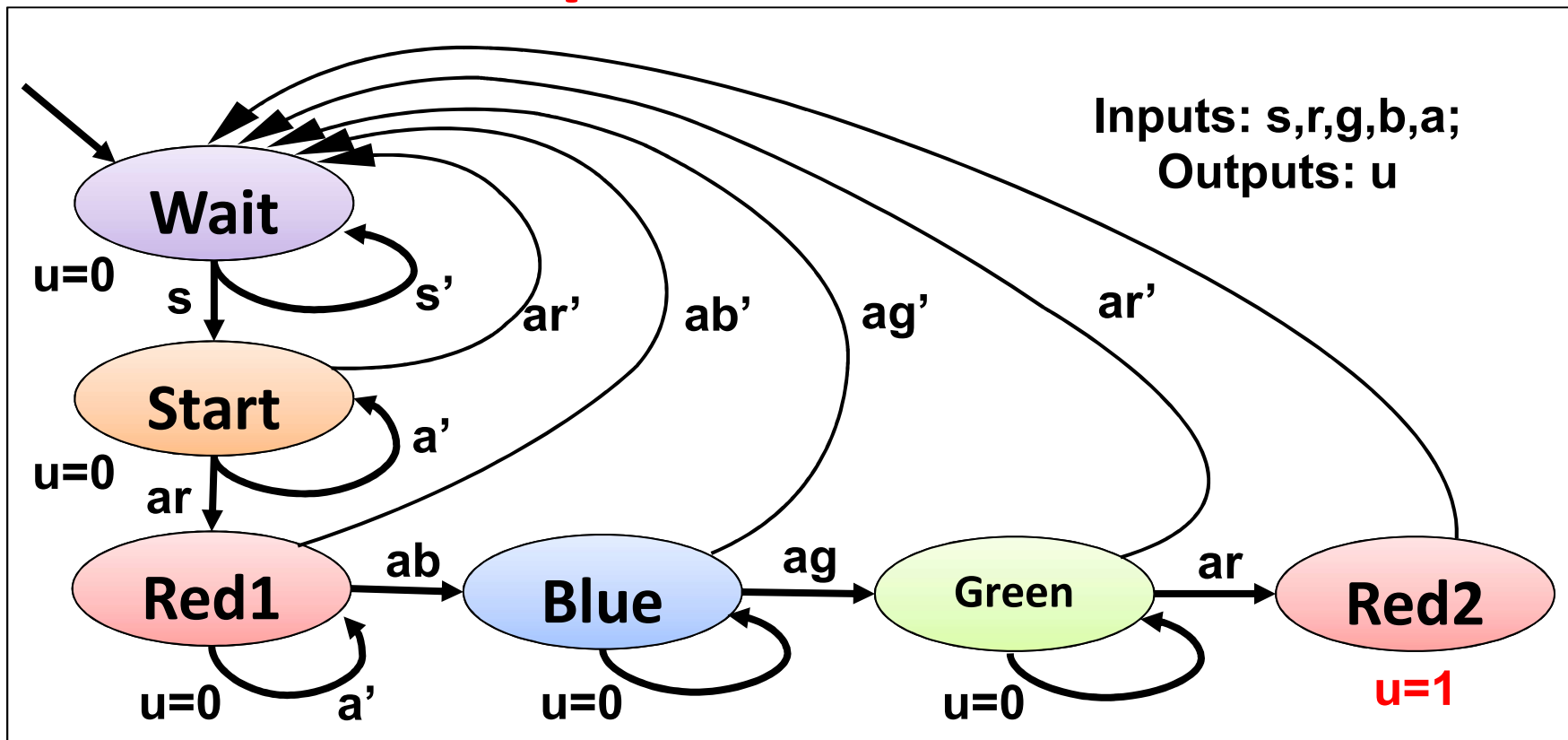


FSM Example 11 : Code Detector



Q: Can you trick this FSM to open the door, without knowing the code?

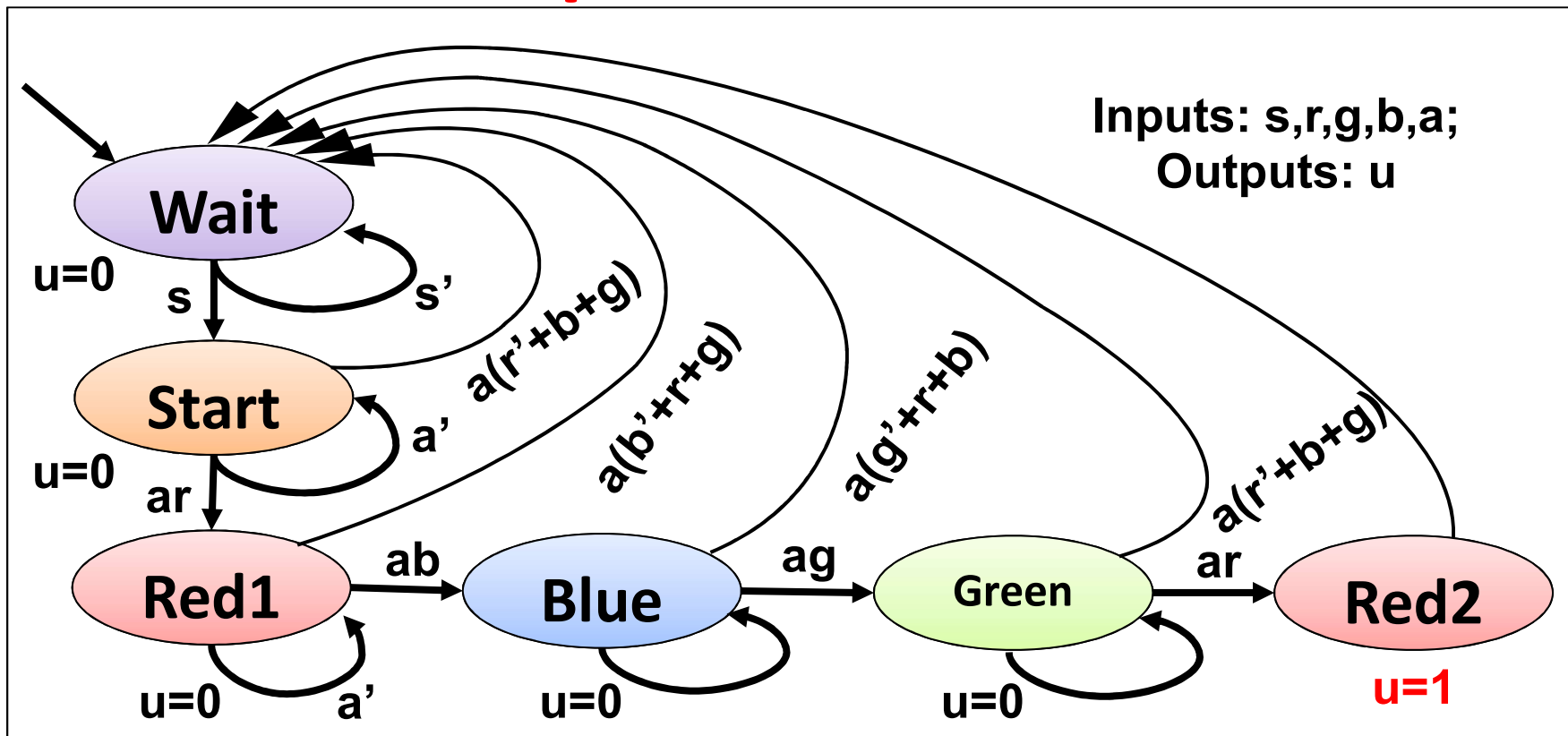
FSM Example 11 : Code Detector



Q: Can you trick this FSM to open the door, without knowing the code?

A: Yes, hold all buttons simultaneously

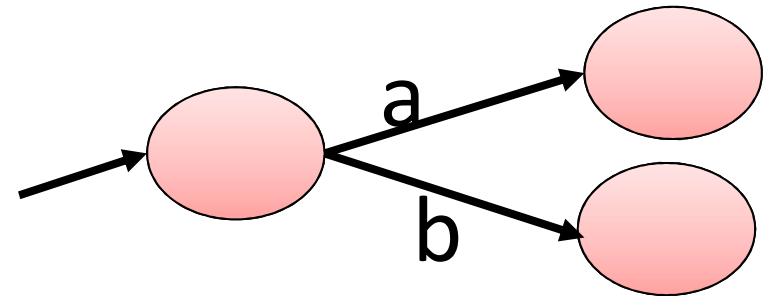
FSM Example 11 : Code Detector



- New transition conditions detect if wrong button pressed, returns to "Wait"
- FSM provides formal, concrete means to accurately define desired behavior

Common Pitfalls Regarding Transition Properties

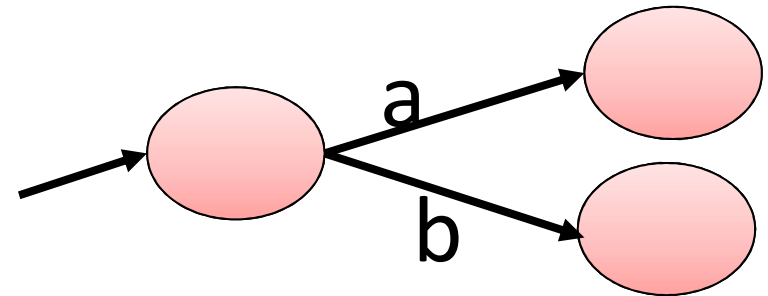
- *Only* one condition should be true
 - For all transitions leaving a state
 - Else, which one?



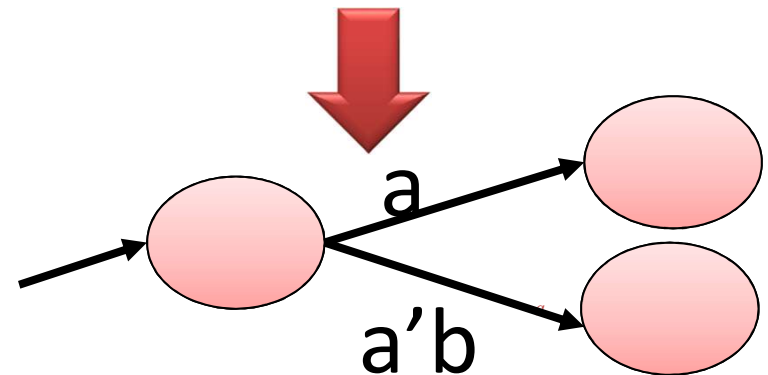
$ab=11$
Next state?

Common Pitfalls Regarding Transition Properties

- *Only* one condition should be true
 - For all transitions leaving a state
 - Else, which one?

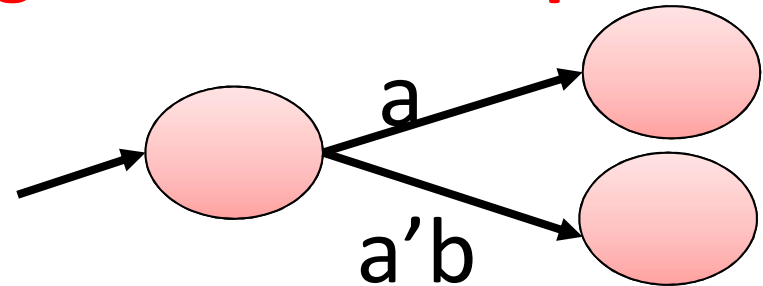


$ab=11$
Next state?



Common Pitfalls Regarding Transition Properties

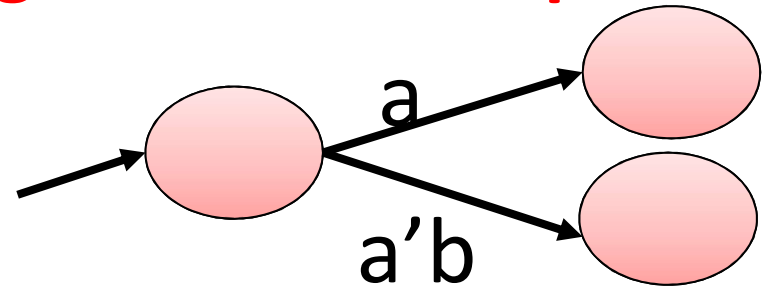
- *One* condition must be true
 - For all transitions leaving a state
 - **Else, where to go?**



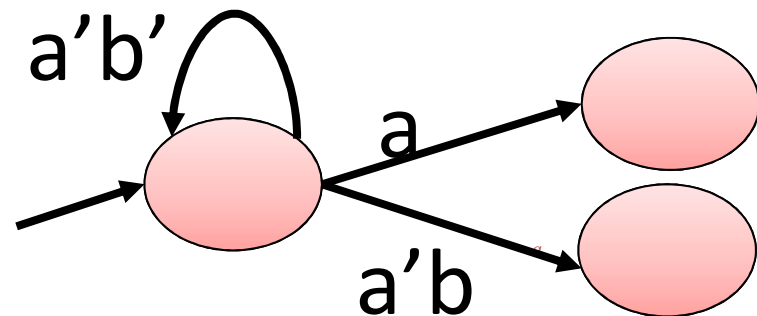
What if $ab=00$?

Common Pitfalls Regarding Transition Properties

- *One* condition must be true
 - For all transitions leaving a state
 - **Else, where to go?**

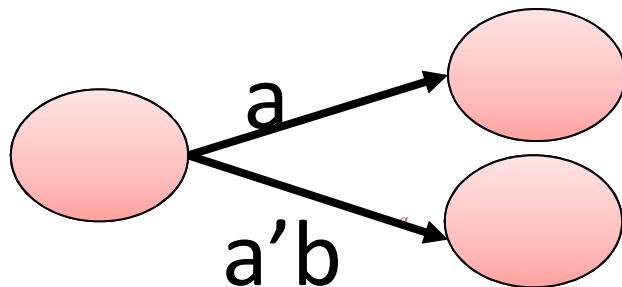


What if $ab=00$?



Verifying Correct Transition Properties

- Can verify using Boolean algebra
 - Only one condition true: AND of each condition pair (for transitions leaving a state) should equal 0
 - $\rightarrow$ proves pair can never simultaneously be true
 - Example

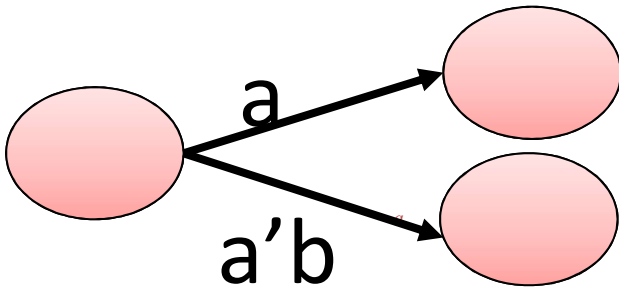


Answer:

$$\begin{aligned} & a * a'b \\ &= (a * a') * b \\ &= 0 * b \\ &= 0 \\ &\text{OK!} \end{aligned}$$

Verifying Correct Transition Properties

- Can verify using Boolean algebra
 - **One condition true: OR of all conditions of transitions leaving a state) should equal 1**
 - $\rightarrow$ proves at least one condition must be true
 - Example



$$\begin{aligned} & a + a'b \\ &= a \cdot (1+b) + a'b = a + ab + a'b \\ &= a + (a+a')b = \mathbf{a + b} \end{aligned}$$

**Fails! Might not be 1 (i.e.,
a=0, b=0)**

Verifying Correct Transition Properties

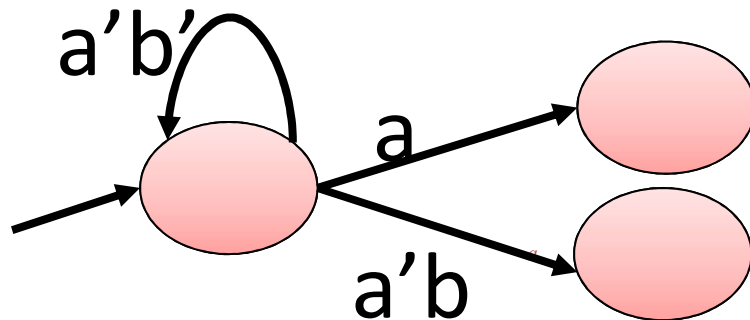
- Can verify using Boolean algebra
 - Only one Condition true : among all pairs of transition from a state
 - One condition true : All the transitions from a state

Verifying Correct Transition Properties

Q: For shown transitions, prove whether:

- Only one condition true (AND of each pair is always 0)
- One condition true (OR of all transitions is always 1)

a



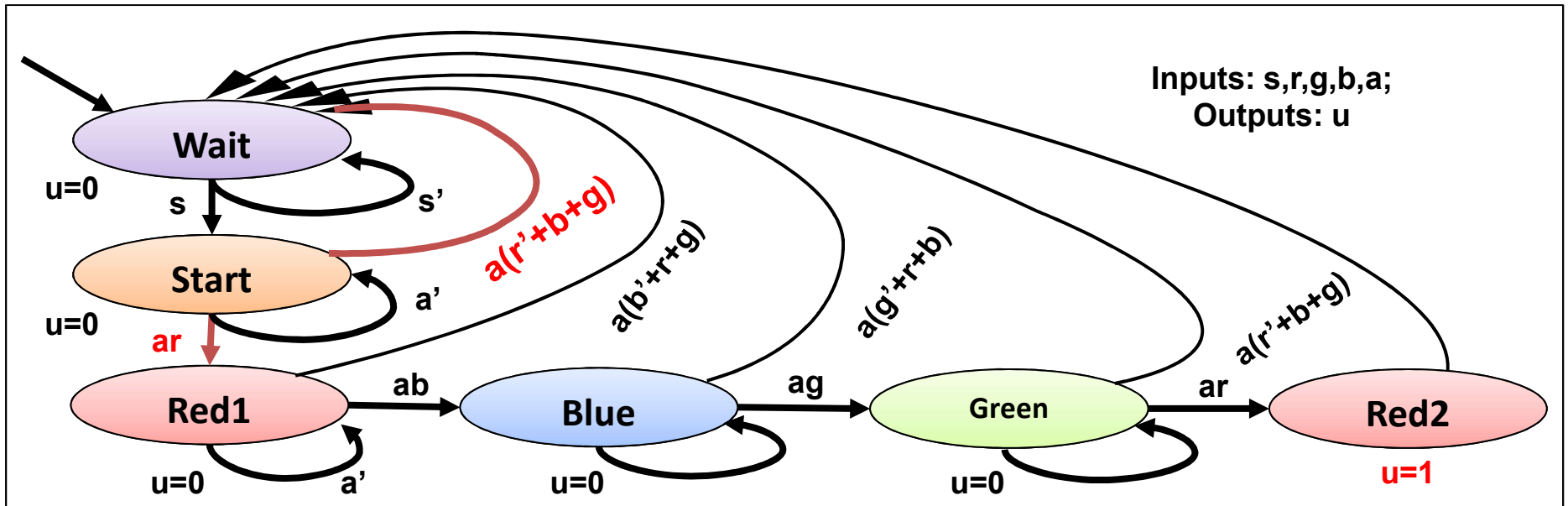
$$a * a'b = 0$$

$$a * a'b' = 0$$

$$a'b * ab' = 0$$

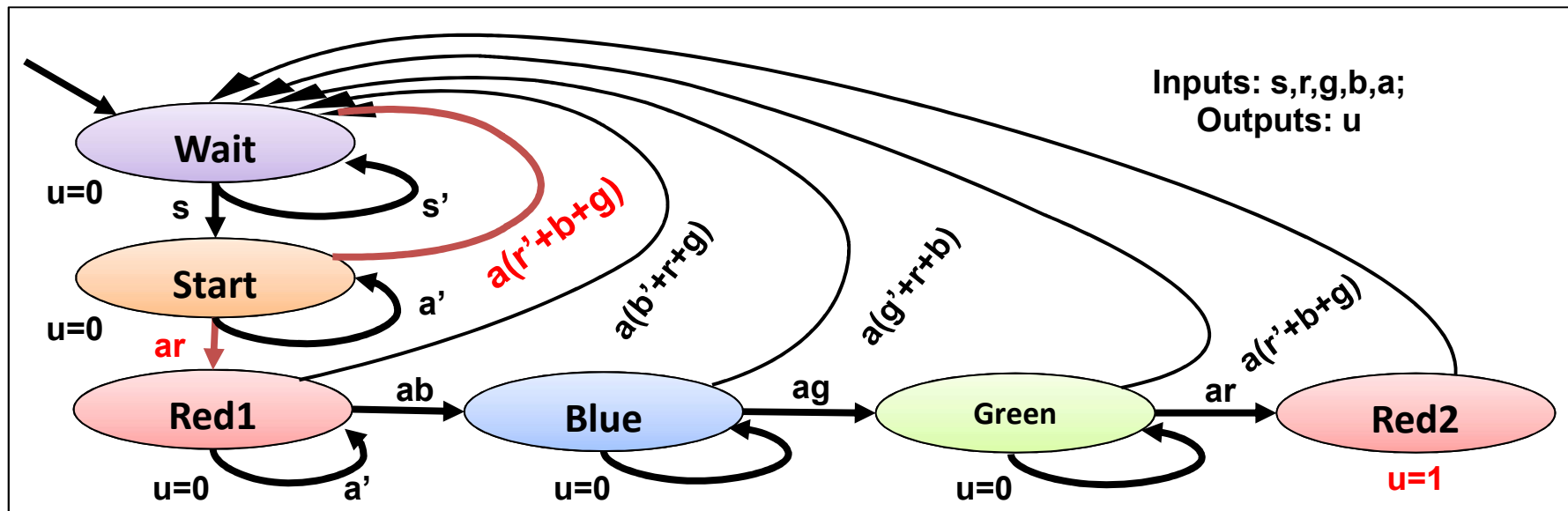
$$a + a'b + a'b' = 1$$

Evidence that Pitfall is Common



- Recall code detector FSM
 - We “fixed” a problem with the transition conditions
 - Do the transitions obey the two required transition properties?
 - Consider transitions of state *Start*, and the “**only one true**” property

Evidence that Pitfall is Common



Consider transitions of state *Start*, and “only one true” property

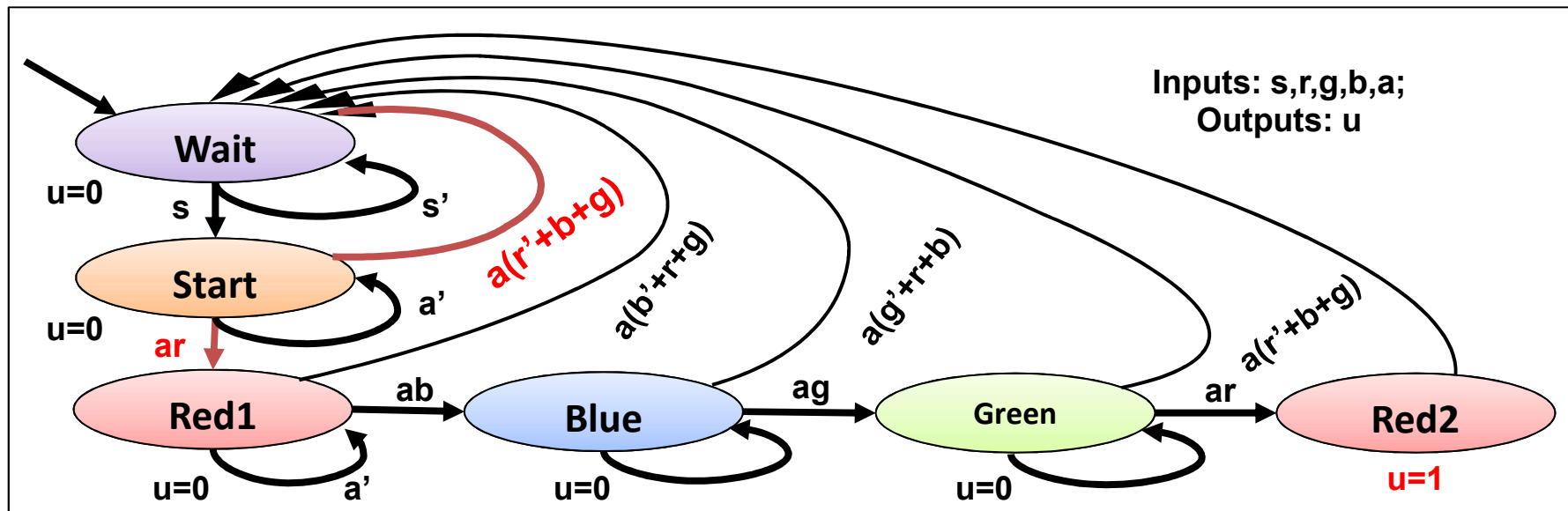
$$ar * a' = 0$$

$$a' * a(r'+b+g) = 0$$

$$ar * a(r'+b+g) = ar(b+g) \text{ // not ZERO}$$

Fails! Means that two of Start's transitions could be true

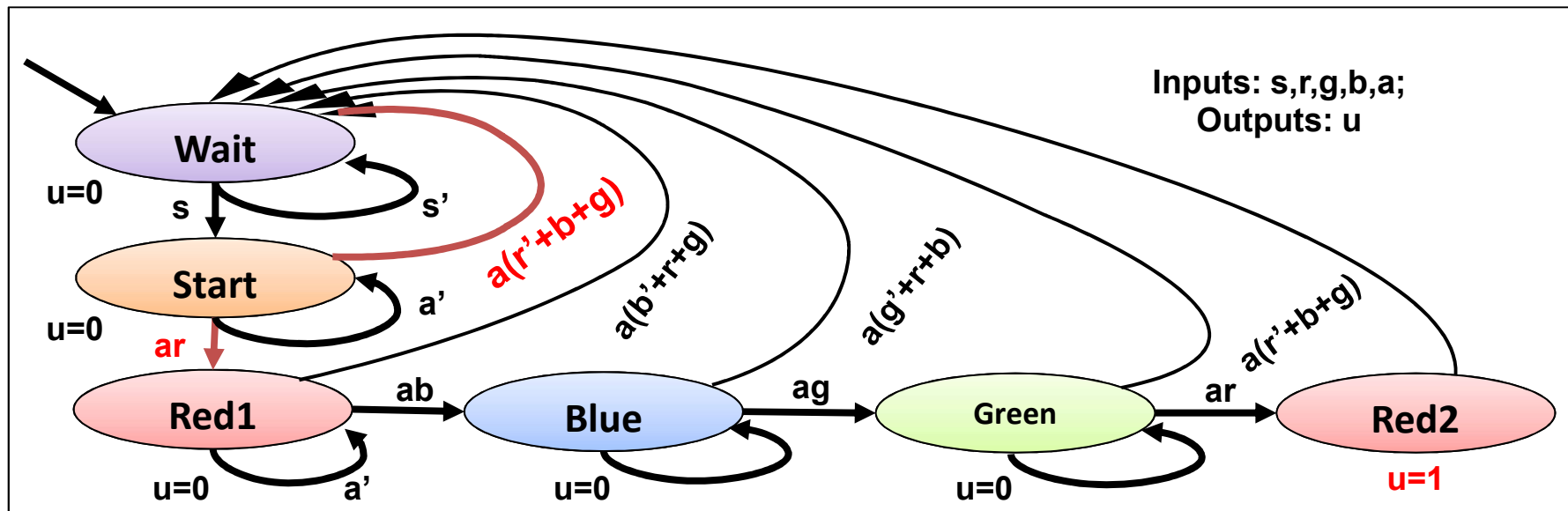
Evidence that Pitfall is Common



$$ar * a(r'+b+g) = ar(b+g) \text{ // not ZERO}$$

Intuitively: press red and blue buttons at same time: **conditions ar, and a(r'+b+g) will both be true. Which one should be taken?**

Evidence that Pitfall is Common



$$ar * a(r'+b+g) = ar(b+g) \text{ // not ZERO}$$

Q: How to solve?

ANS: $arb'g'$
(likewise for ab, ag, ar)

$$arb'g' * a(r'+b+g) = 0$$

Evidence that Pitfall is Common

